GitHub Repositories:
https://github.com/viktiv-dev/ar-project
https://github.com/viktiv-dev/vr-project

Personal Reflection

Working on both the AR and VR projects provided valuable insight into the practical challenges of XR development beyond the theoretical concepts discussed in class. While the projects shared a common theme and reused assets, the development experience for each highlighted different strengths and limitations of current XR tools.

One of the main challenges during the AR project was the reliance on Vuforia for image tracking. While Vuforia offers strong tracking capabilities and useful evaluation tools, it lacks native Linux support, even through Unity and I ended up spending too much time on that issues, which significantly impacted my development workflow. This limitation meant that certain parts of the project had to be tested in constrained environments or postponed until access to a compatible system was available. As a result, much of the early development relied heavily on simulation rather than direct testing on physical devices.

Similarly, for both AR and VR, a large portion of development was done using simulation tools rather than real hardware. In AR, camera simulation and mock tracking were essential for iterating on logic without constant device deployment. In VR, Unity's XR Device Simulator made it possible to test interactions, locomotion, and object manipulation without access to a physical headset. While simulation is extremely useful for rapid iteration, it also became clear that it cannot fully replicate real-world behavior, particularly when it comes to comfort and physical interaction.

Unity was chosen for both projects primarily due to familiarity and its well-established XR ecosystem. Knowing the engine and its user interface allowed me to focus on solving XR-specific problems rather than learning a new development environment. Unity's editor tools, prefab system, and XR frameworks made it easier to prototype ideas quickly and adjust designs based on testing feedback. The availability of packages such as AR Foundation and the XR Interaction Toolkit also helped maintain consistency across projects.

Developing XR applications introduced several challenges that are less common in traditional software development. These included managing spatial scale, ensuring comfortable locomotion, designing intuitive interactions and handling tracking reliability. Small issues such as incorrect scale, unstable physics, or poorly designed input had a much larger impact on user experience than they would in a desktop application.

Overall, these projects demonstrated that XR development is as much about design and constraints as it is about technical implementation. Working through platform limitations, simulation based testing and interaction challenges provided a more realistic understanding of what it takes to build usable XR exxperiences. Both projects contributed to a stronger appreciation of XR as a medium and highlighted areas for further improvement and learning in future work.